PRACTICAL FILE

Open Source Web Programming using PHP

(Paper 1T3)

First Year M.C.A. — Semester I (CBCS)

RTM Nagpur University

Name: _____

Roll No: _____

INDEX

Sr. No.	Practical	Date	Sign
1	Write a PHP program that Process a Form.		
2	Write a PHP script to create, retrieve and deleting the Cookies data.		
3	Write a PHP script to create, retrieve and deleting the Session data.		
4	Write a PHP Script to define the Class and instantiate the Object of that class.		
5	Write a PHP program to create a table, select data and show into table format in MySQL.		
6	Write a PHP program to update table using MySQL.		
7	Write a PHP program to insert XML file in PHP.		
8	Write a program in PHP that include another file into current file using include and require function.		
9	Write a program in PHP to demonstrate the Types of Arrays.		
10	Write a program in PHP to search an element in an array.		
11	Write a program in PHP to create and print the elements of an array using different Array Printing functions.		
12	Write a program in PHP that Sorts the given Array using different array sorting functions.		
13	Write a program in PHP that Chunkify the given string.		
14	Write a PHP script to check that email id is valid or not.		
15	Write a PHP function calculateGST($amount) that returns the GST (18%) of the given amount. Call the function with a sample value and display the result.		

Practical No. 1

Aim: Write a PHP program that Process a Form.

Algorithm:

Step 1: Start.

Step 2: Display an HTML form with fields Name, Email and Course and a Submit button (method = POST).

Step 3: User enters the data and clicks Submit.

Step 4: Check whether the form is submitted using isset($_POST['submit']).

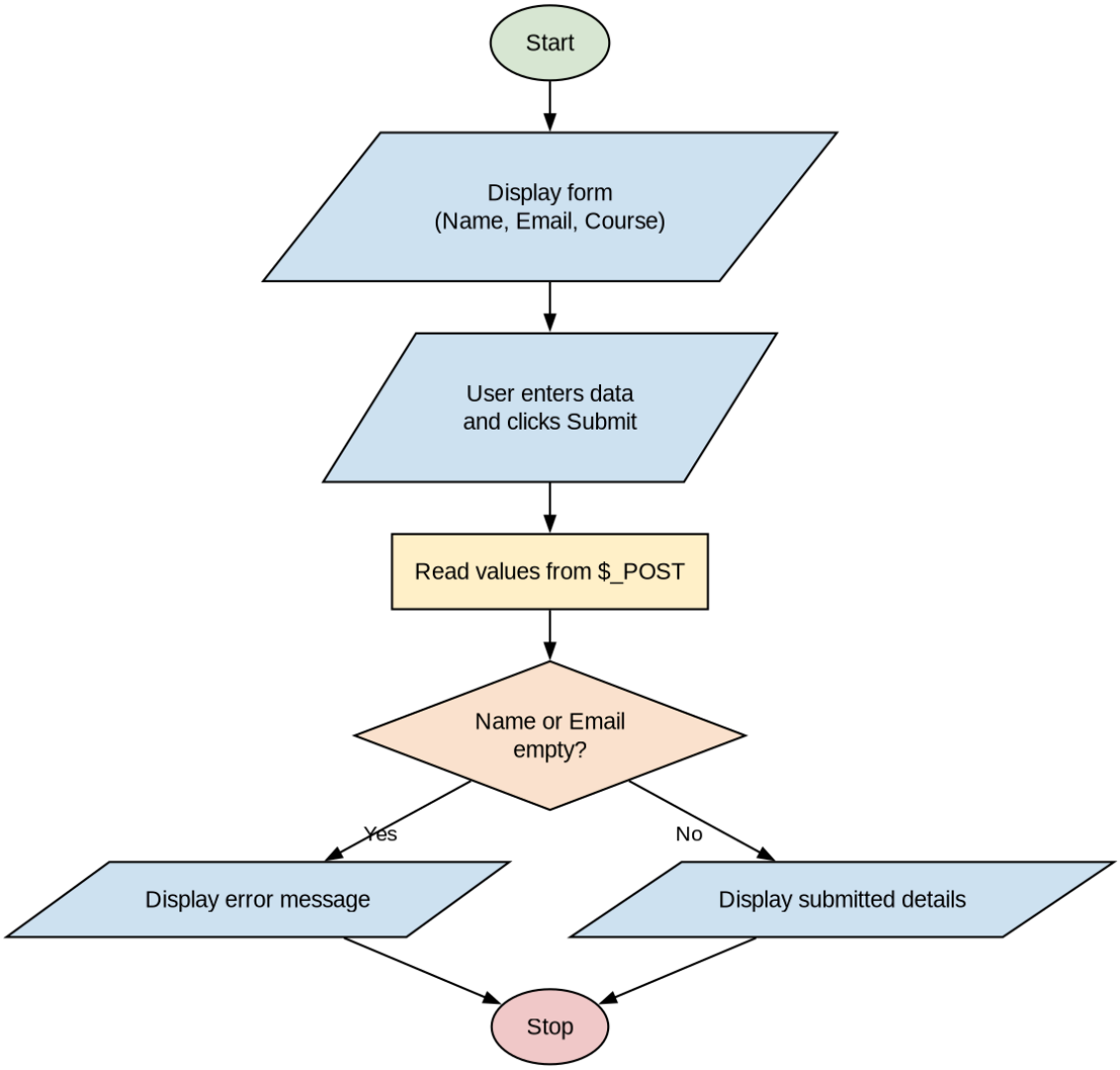
Step 5: Read the values from $_POST and clean them using trim() and htmlspecialchars().

Step 6: If Name or Email is empty, display an error message.

Step 7: Otherwise display the submitted details.

Step 8: Stop.

Flowchart:


Start
Display form
(Name, Email, Course)
User enters data
and clicks Submit
Read values from $_POST
Name or Email
empty?
Yes
No
Display error message
Display submitted details
Stop

Program:

```
<!DOCTYPE html>
<html>
<head><title>Form Processing</title></head>
<body>
<h2>Student Registration Form</h2>
<form method="post" action="">
  Name: <input type="text" name="name"><br><br>
  Email: <input type="text" name="email"><br><br>
  Course:
  <select name="course">
    <option>MCA</option>
    <option>MBA</option>
  </select><br><br>
  <input type="submit" name="submit" value="Submit">
</form>

<?php
if (isset($_POST['submit'])) {
    $name   = htmlspecialchars(trim($_POST['name']));
    $email  = htmlspecialchars(trim($_POST['email']));
    $course = $_POST['course'];

    if ($name == "" || $email == "") {
        echo "<p style='color:red'>All fields are required!</p>";
    } else {
        echo "<h3>Submitted Details</h3>";
        echo "Name   : $name <br>";
        echo "Email  : $email <br>";
        echo "Course : $course <br>";
    }
}
?>
</body>
</html>
```

Output:

```
Student Registration Form
Name:   [Rahul Sharma        ]
Email:  [rahul@gmail.com     ]
Course: [MCA v]
[Submit]

Submitted Details
Name   : Rahul Sharma
Email  : rahul@gmail.com
Course : MCA
```

Practical No. 2

Aim: Write a PHP script to create, retrieve and deleting the Cookies data.

Algorithm:

Step 1: Start.

Step 2: Display three links: Create Cookie, Retrieve Cookie and Delete Cookie.

Step 3: Read the action selected by the user from $_GET['action'].

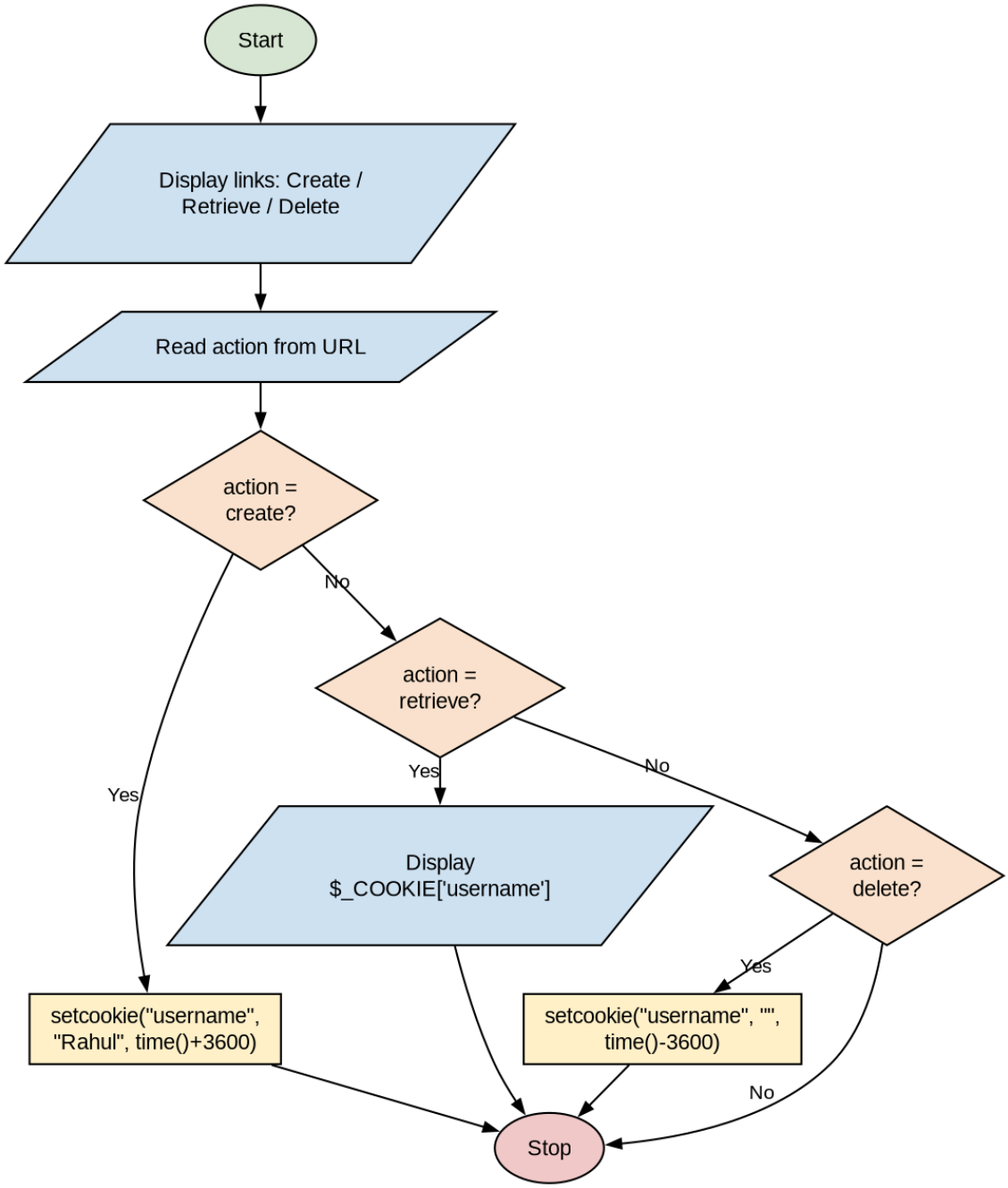
Step 4: If action = create, create the cookie using setcookie("username", "Rahul", time() + 3600).

Step 5: If action = retrieve, check isset($_COOKIE['username']); if set display its value, else display "Cookie not found".

Step 6: If action = delete, delete the cookie by calling setcookie() with an expiry time in the past.

Step 7: Stop.

Flowchart:


Start
Display links: Create / Retrieve / Delete
Read action from URL
action = create?
No
Yes
action = retrieve?
Yes
No
Display $_COOKIE['username']
action = delete?
Yes
No
setcookie("username", "Rahul", time()+3600)
setcookie("username", "", time()-3600)
Stop

Program:

```
<?php
$msg = "";
if (isset($_GET['action'])) {
    if ($_GET['action'] == "create") {
        // Create cookie valid for 1 hour
        setcookie("username", "Rahul", time() + 3600);
        $msg = "Cookie 'username' created successfully.";
    } elseif ($_GET['action'] == "delete") {
        // Delete cookie by setting expiry time in the past
        setcookie("username", "", time() - 3600);
        $msg = "Cookie 'username' deleted successfully.";
    }
}
?>
<html>
<body>
<h2>Cookie Demo</h2>
<a href="?action=create">Create Cookie</a> |
<a href="?action=retrieve">Retrieve Cookie</a> |
<a href="?action=delete">Delete Cookie</a>
<hr>
<?php
echo $msg;
if (isset($_GET['action']) && $_GET['action'] == "retrieve") {
    if (isset($_COOKIE['username'])) {
        echo "Cookie value of username: " . $_COOKIE['username'];
    } else {
        echo "Cookie not found.";
    }
}
?>
</body>
</html>
```

Output:

```
Cookie Demo
Create Cookie | Retrieve Cookie | Delete Cookie
-----------------------------------------------
(Click Create Cookie)
Cookie 'username' created successfully.

(Click Retrieve Cookie)
Cookie value of username: Rahul

(Click Delete Cookie)
Cookie 'username' deleted successfully.

(Click Retrieve Cookie again)
Cookie not found.
```

Practical No. 3

Aim: Write a PHP script to create, retrieve and deleting the Session data.

Algorithm:

Step 1: Start.

Step 2: Start the session using session_start().

Step 3: Display three links: Create Session, Retrieve Session and Delete Session.

Step 4: Read the action selected by the user from $_GET['action'].

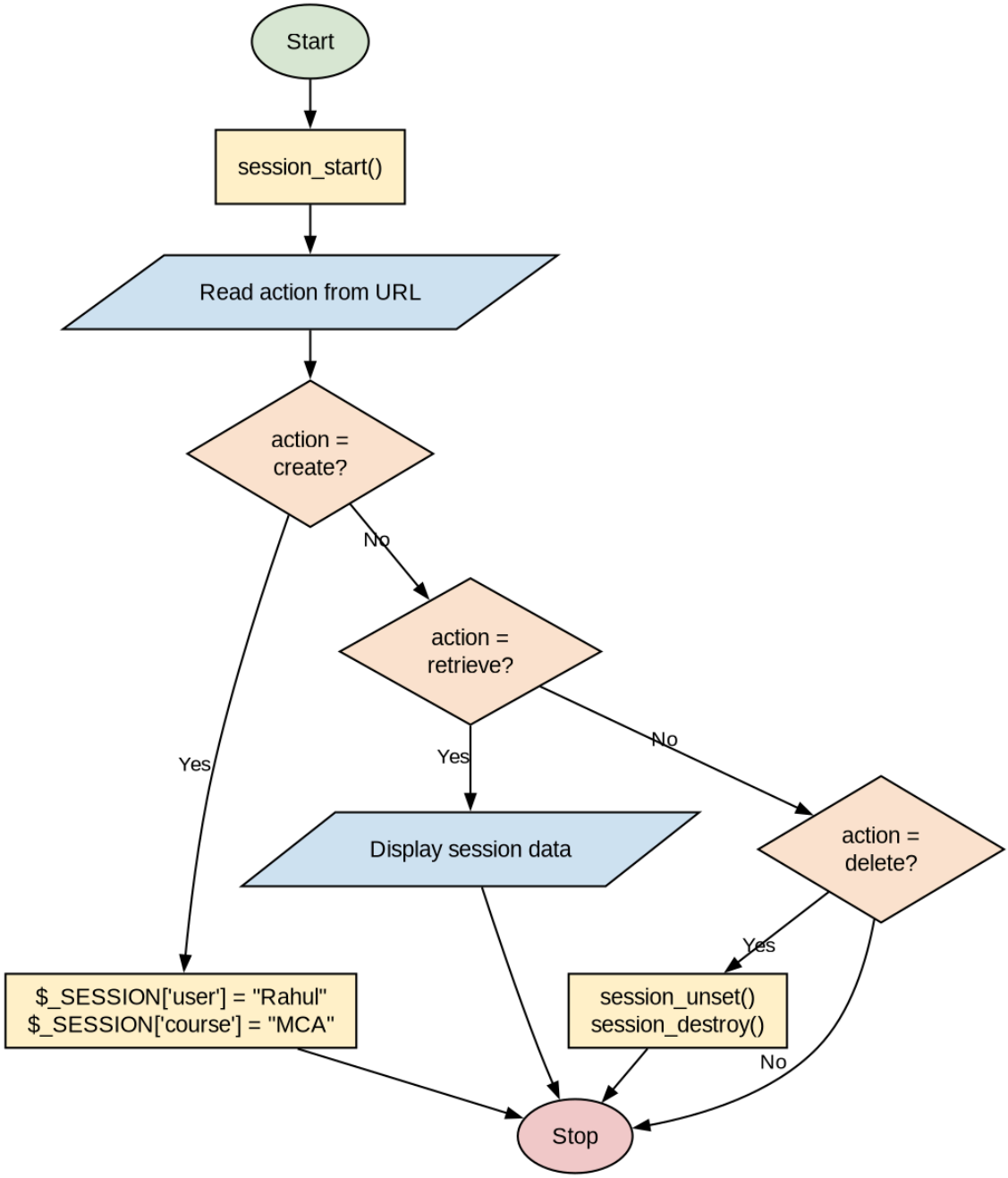
Step 5: If action = create, store data in $_SESSION['user'] and $_SESSION['course'].

Step 6: If action = retrieve, check isset($_SESSION['user']); if set display the session data, else display "No session data found".

Step 7: If action = delete, remove the data using session_unset() and session_destroy().

Step 8: Stop.

Flowchart:


Start
session_start()
Read action from URL
action = create?
No
Yes
action = retrieve?
Yes
No
Display session data
action = delete?
Yes
No
$_SESSION['user'] = "Rahul"
$_SESSION['course'] = "MCA"
session_unset()
session_destroy()
Stop

Program:

```
<?php
session_start();
?>
<html>
<body>
<h2>Session Demo</h2>
<a href="?action=create">Create Session</a> |
<a href="?action=retrieve">Retrieve Session</a> |
<a href="?action=delete">Delete Session</a>
<hr>
<?php
if (isset($_GET['action'])) {
    $action = $_GET['action'];

    if ($action == "create") {
        $_SESSION['user']   = "Rahul";
        $_SESSION['course'] = "MCA";
        echo "Session data created successfully.";
    } elseif ($action == "retrieve") {
        if (isset($_SESSION['user'])) {
            echo "User   : " . $_SESSION['user'] . "<br>";
            echo "Course : " . $_SESSION['course'];
        } else {
            echo "No session data found.";
        }
    } elseif ($action == "delete") {
        session_unset();     // remove all session variables
        session_destroy();   // destroy the session
        echo "Session data deleted successfully.";
    }
}
?>
</body>
</html>
```

Output:

```
Session Demo
Create Session | Retrieve Session | Delete Session
-----------------------------------------------
(Click Create Session)
Session data created successfully.

(Click Retrieve Session)
User   : Rahul
Course : MCA

(Click Delete Session)
Session data deleted successfully.

(Click Retrieve Session again)
No session data found.
```

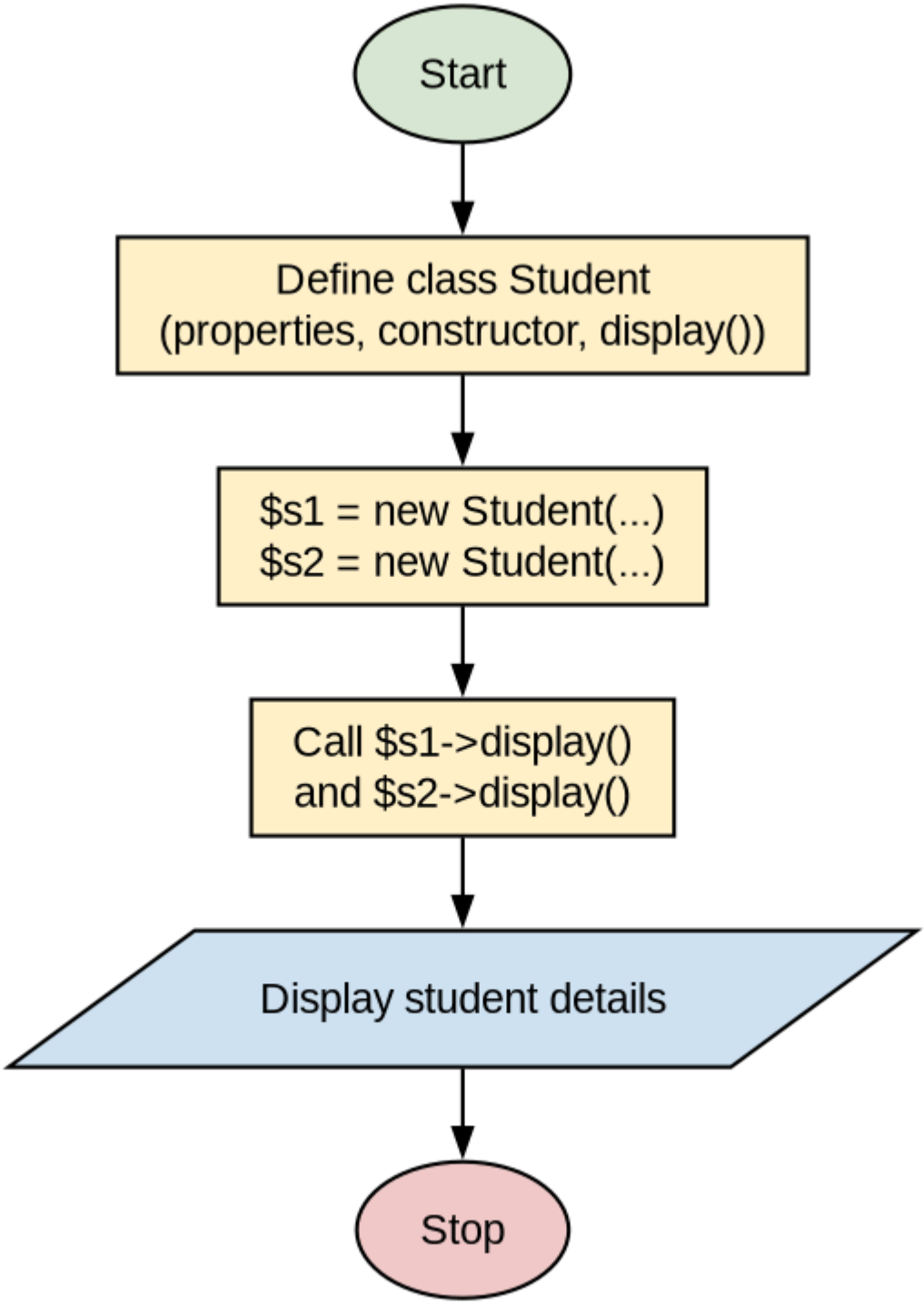
Practical No. 4

Aim: Write a PHP Script to define the Class and instantiate the Object of that class.

Algorithm:

Step 1: Start.
Step 2: Define a class Student with properties $name, $roll and $course.
Step 3: Define a constructor __construct() to initialize the properties.
Step 4: Define a method display() to print the student details.
Step 5: Create objects $s1 and $s2 of class Student using the new keyword.
Step 6: Call the display() method using each object.
Step 7: Stop.

Flowchart:


Start
Define class Student
(properties, constructor, display())
$s1 = new Student(...)
$s2 = new Student(...)
Call $s1->display()
and $s2->display()
Display student details
Stop

Program:

```
<?php
class Student {
    public $name;
    public $roll;
    public $course;

    // Constructor
    function __construct($name, $roll, $course) {
        $this->name   = $name;
        $this->roll   = $roll;
        $this->course = $course;
    }

    // Method
    function display() {
        echo "Roll No : " . $this->roll . "<br>";
        echo "Name    : " . $this->name . "<br>";
        echo "Course  : " . $this->course . "<br><br>";
    }
}

// Instantiate objects
$s1 = new Student("Rahul", 101, "MCA");
$s2 = new Student("Priya", 102, "MCA");

echo "<h3>Student Details</h3>";
$s1->display();
$s2->display();
?>
```

Output:

```
Student Details
Roll No : 101
Name    : Rahul
Course  : MCA

Roll No : 102
Name    : Priya
Course  : MCA
```

Practical No. 5

Aim: Write a PHP program to create a table, select data and show into table format in MySQL.

Pre-requisite: Start Apache and MySQL (XAMPP/WAMP) and create the database using: CREATE DATABASE college;

Algorithm:

Step 1: Start.

Step 2: Connect to the MySQL database "college" using mysqli_connect().

Step 3: If the connection fails, display an error and stop.

Step 4: Create the table student (id, name, course, marks) using CREATE TABLE IF NOT EXISTS.

Step 5: Insert sample records into the table.

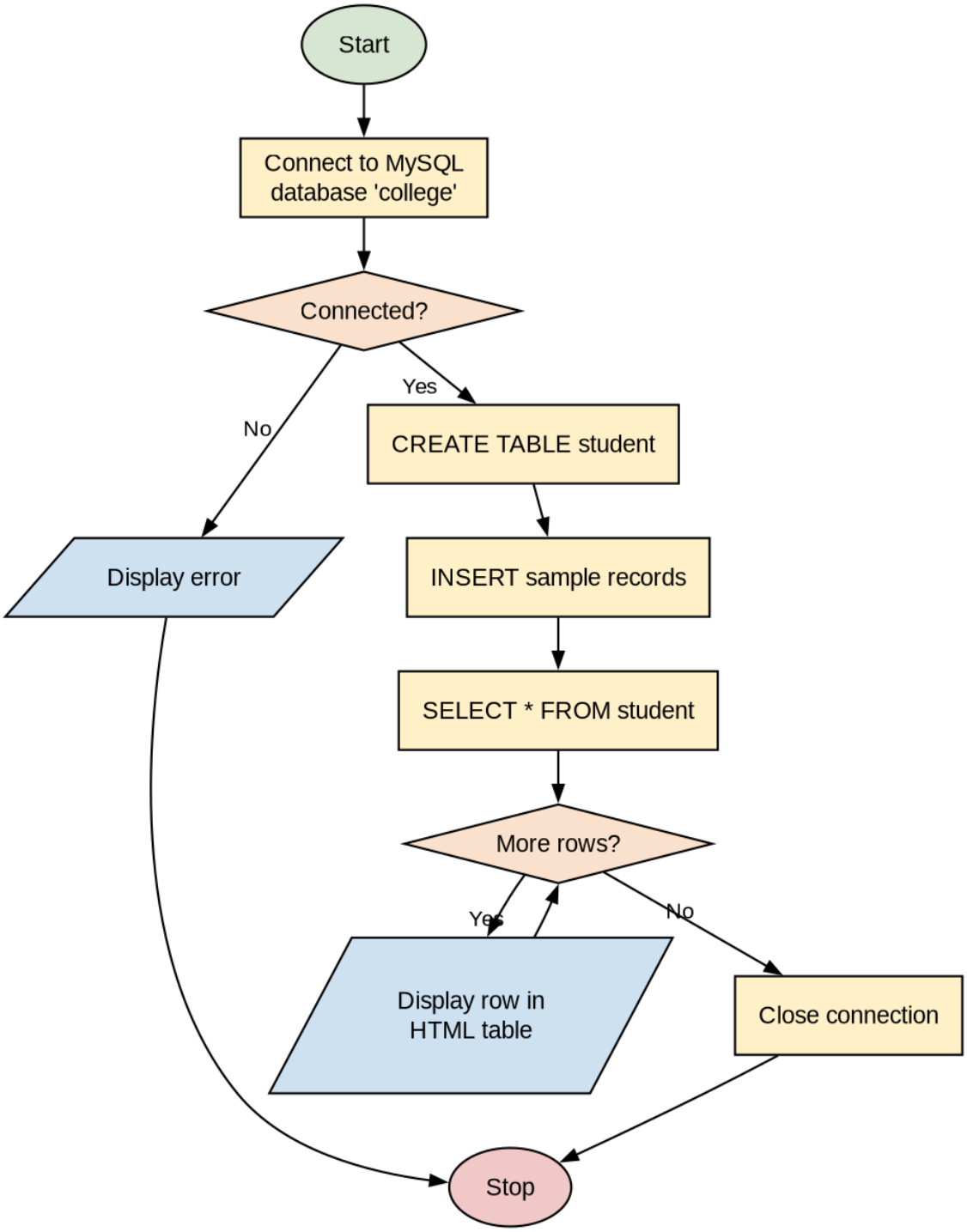
Step 6: Execute the query SELECT * FROM student.

Step 7: Fetch each row using mysqli_fetch_assoc() and display it as a row of an HTML table.

Step 8: Close the connection.

Step 9: Stop.

Flowchart:


Start
Connect to MySQL database 'college'
Connected?
No
Yes
Display error
CREATE TABLE student
INSERT sample records
SELECT * FROM student
More rows?
Yes
No
Display row in HTML table
Close connection
Stop

Program:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "college");
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Create table
$sql = "CREATE TABLE IF NOT EXISTS student (
            id     INT PRIMARY KEY,
            name   VARCHAR(50),
            course VARCHAR(20),
            marks  INT
        )";
if (mysqli_query($conn, $sql)) {
    echo "Table 'student' created successfully.<br><br>";
}

// Insert sample records
mysqli_query($conn, "INSERT IGNORE INTO student VALUES
    (1, 'Rahul', 'MCA', 82),
    (2, 'Priya', 'MCA', 90),
    (3, 'Amit',  'MCA', 76)");

// Select and display data in table format
$result = mysqli_query($conn, "SELECT * FROM student");

echo "<table border='1' cellpadding='6'>";
echo "<tr><th>ID</th><th>Name</th><th>Course</th><th>Marks</th></tr>";
while ($row = mysqli_fetch_assoc($result)) {
    echo "<tr>";
    echo "<td>" . $row['id'] . "</td>";
    echo "<td>" . $row['name'] . "</td>";
    echo "<td>" . $row['course'] . "</td>";
    echo "<td>" . $row['marks'] . "</td>";
    echo "</tr>";
}
echo "</table>";

mysqli_close($conn);
?>
```

Output:

```
Table 'student' created successfully.

+----+-------+--------+-------+
| ID | Name  | Course | Marks |
+----+-------+--------+-------+
| 1  | Rahul | MCA    | 82    |
| 2  | Priya | MCA    | 90    |
| 3  | Amit  | MCA    | 76    |
+----+-------+--------+-------+
```

Practical No. 6

Aim: Write a PHP program to update table using MySQL.

Uses the student table created in Practical 5.

Algorithm:

Step 1: Start.

Step 2: Connect to the MySQL database "college".

Step 3: Set the id of the record to update and the new marks.

Step 4: Execute the query UPDATE student SET marks = new_marks WHERE id = id.

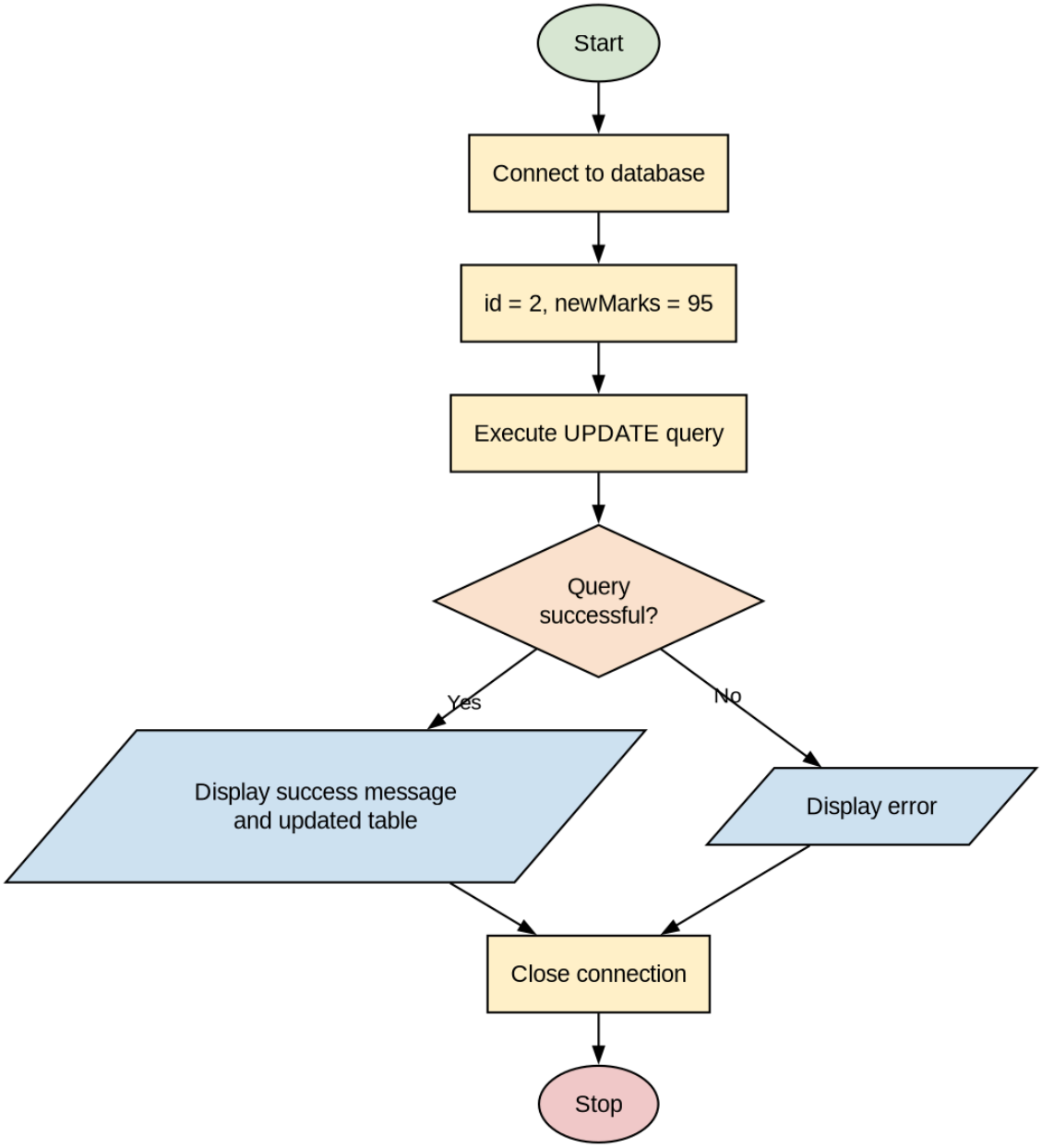
Step 5: If the query is successful, display the success message and the number of rows affected; otherwise display the error.

Step 6: Select all records and display the updated table.

Step 7: Close the connection.

Step 8: Stop.

Flowchart:


Start
Connect to database
id = 2, newMarks = 95
Execute UPDATE query
Query successful?
Yes
No
Display success message and updated table
Display error
Close connection
Stop

Program:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "college");
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

$id = 2;
$newMarks = 95;

$sql = "UPDATE student SET marks = $newMarks WHERE id = $id";
if (mysqli_query($conn, $sql)) {
    echo "Record updated successfully.<br>";
    echo "Rows affected: " . mysqli_affected_rows($conn) . "<br><br>";
} else {
    echo "Error updating record: " . mysqli_error($conn);
}

// Display updated table
$result = mysqli_query($conn, "SELECT * FROM student");
echo "<table border='1' cellpadding='6'>";
echo "<tr><th>ID</th><th>Name</th><th>Course</th><th>Marks</th></tr>";
while ($row = mysqli_fetch_assoc($result)) {
    echo "<tr><td>" . $row['id'] . "</td><td>" . $row['name'] .
         "</td><td>" . $row['course'] . "</td><td>" . $row['marks'] . "</td></tr>";
}
echo "</table>";

mysqli_close($conn);
?>
```

Output:

```
Record updated successfully.
Rows affected: 1

+----+-------+--------+-------+
| ID | Name  | Course | Marks |
+----+-------+--------+-------+
| 1  | Rahul | MCA    | 82    |
| 2  | Priya | MCA    | 95    |
| 3  | Amit  | MCA    | 76    |
+----+-------+--------+-------+
```

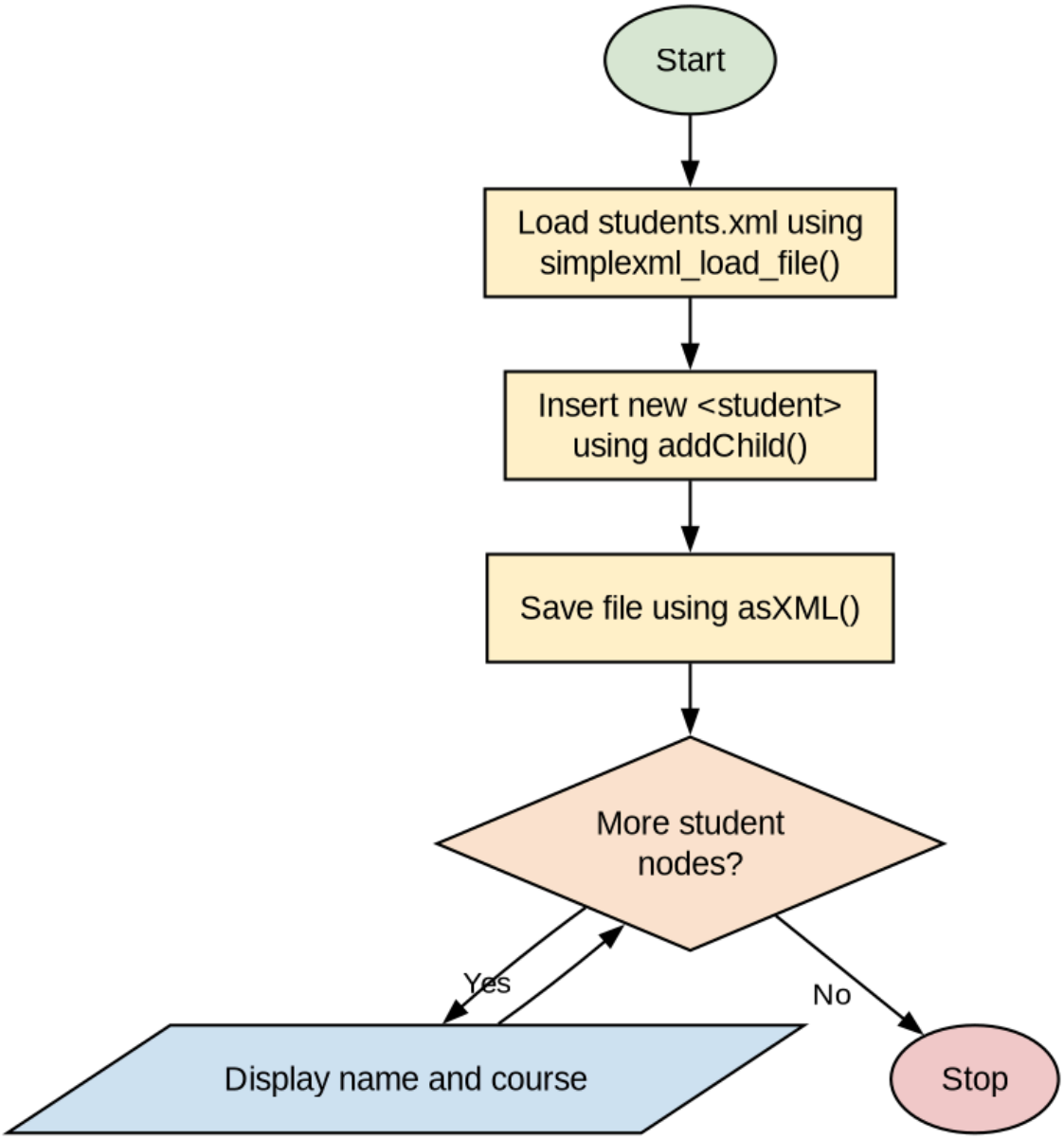
Practical No. 7

Aim: Write a PHP program to insert XML file in PHP.

Algorithm:

Step 1: Start.
Step 2: Create an XML file students.xml containing student records.
Step 3: Load the XML file into PHP using simplexml_load_file().
Step 4: Insert a new <student> element using addChild() and save the file using asXML().
Step 5: Traverse each <student> node using foreach and display its name and course.
Step 6: Stop.

Flowchart:


Start
Load students.xml using simplexml_load_file()
Insert new <student> using addChild()
Save file using asXML()
More student nodes?
Yes
No
Display name and course
Stop

Program:

```
students.xml
-------------
<?xml version="1.0" encoding="UTF-8"?>
<students>
    <student>
        <name>Rahul</name>
        <course>MCA</course>
    </student>
    <student>
        <name>Priya</name>
        <course>MCA</course>
    </student>
</students>

xml_demo.php
------------
<?php
// Load XML file into PHP
$xml = simplexml_load_file("students.xml") or die("Error: Cannot load XML file");

// Insert a new record into the XML
$new = $xml->addChild("student");
$new->addChild("name", "Amit");
$new->addChild("course", "MBA");
$xml->asXML("students.xml");

// Display all records
echo "<h3>Students from XML file</h3>";
foreach ($xml->student as $s) {
    echo "Name: " . $s->name . " | Course: " . $s->course . "<br>";
}
?>
```

Output:

```
Students from XML file
Name: Rahul | Course: MCA
Name: Priya | Course: MCA
Name: Amit | Course: MBA
```

Practical No. 8

Aim: Write a program in PHP that include another file into current file using include and require function.

Algorithm:

Step 1: Start.

Step 2: Create header.php containing the page heading.

Step 3: Create footer.php containing the footer text.

Step 4: In main.php, include header.php using include().

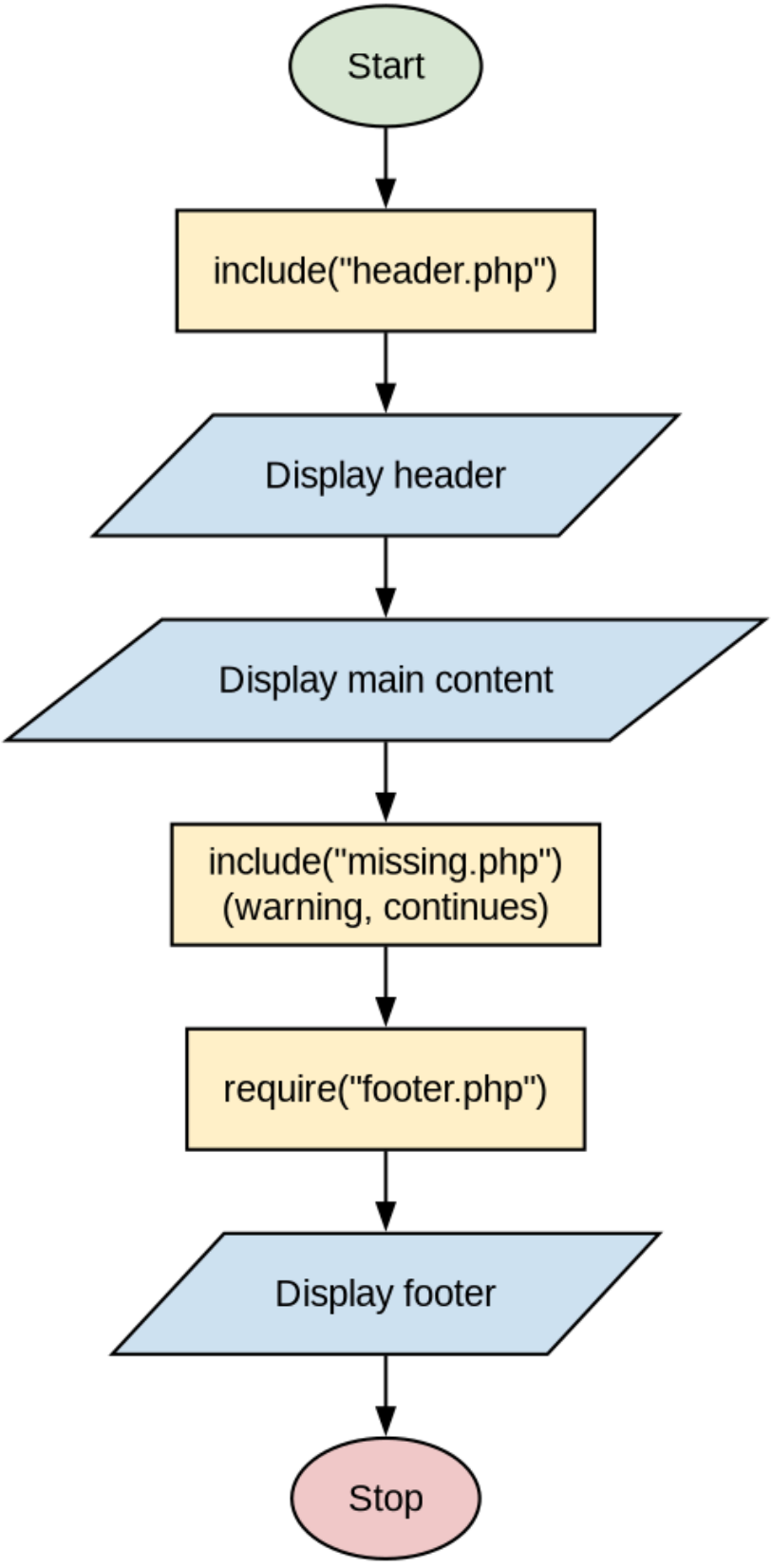
Step 5: Display the main content of the page.

Step 6: Include a non-existing file using include() — PHP gives a warning and continues execution.

Step 7: Include footer.php using require() (require() stops the script with a fatal error if the file is missing).

Step 8: Stop.

Flowchart:


Start
include("header.php")
Display header
Display main content
include("missing.php")
(warning, continues)
require("footer.php")
Display footer
Stop

Program:

```
header.php
----------
<?php
echo "<h2>Welcome to PHP Practical</h2><hr>";
?>

footer.php
----------
<?php
echo "<hr><p>&copy; 2026 MCA Department</p>";
?>

main.php
--------
<?php
include("header.php");

echo "<p>This is the main content of the page.</p>";

// include() gives only a warning if the file is missing
include("missing.php");
echo "<p>Script continues after include() warning.</p>";

// require() gives a fatal error if the file is missing
require("footer.php");
?>
```

Output:

```
Welcome to PHP Practical
----------------------------------------
This is the main content of the page.

Warning: include(missing.php): Failed to open stream:
No such file or directory in main.php on line 7

Script continues after include() warning.
----------------------------------------
© 2026 MCA Department
```

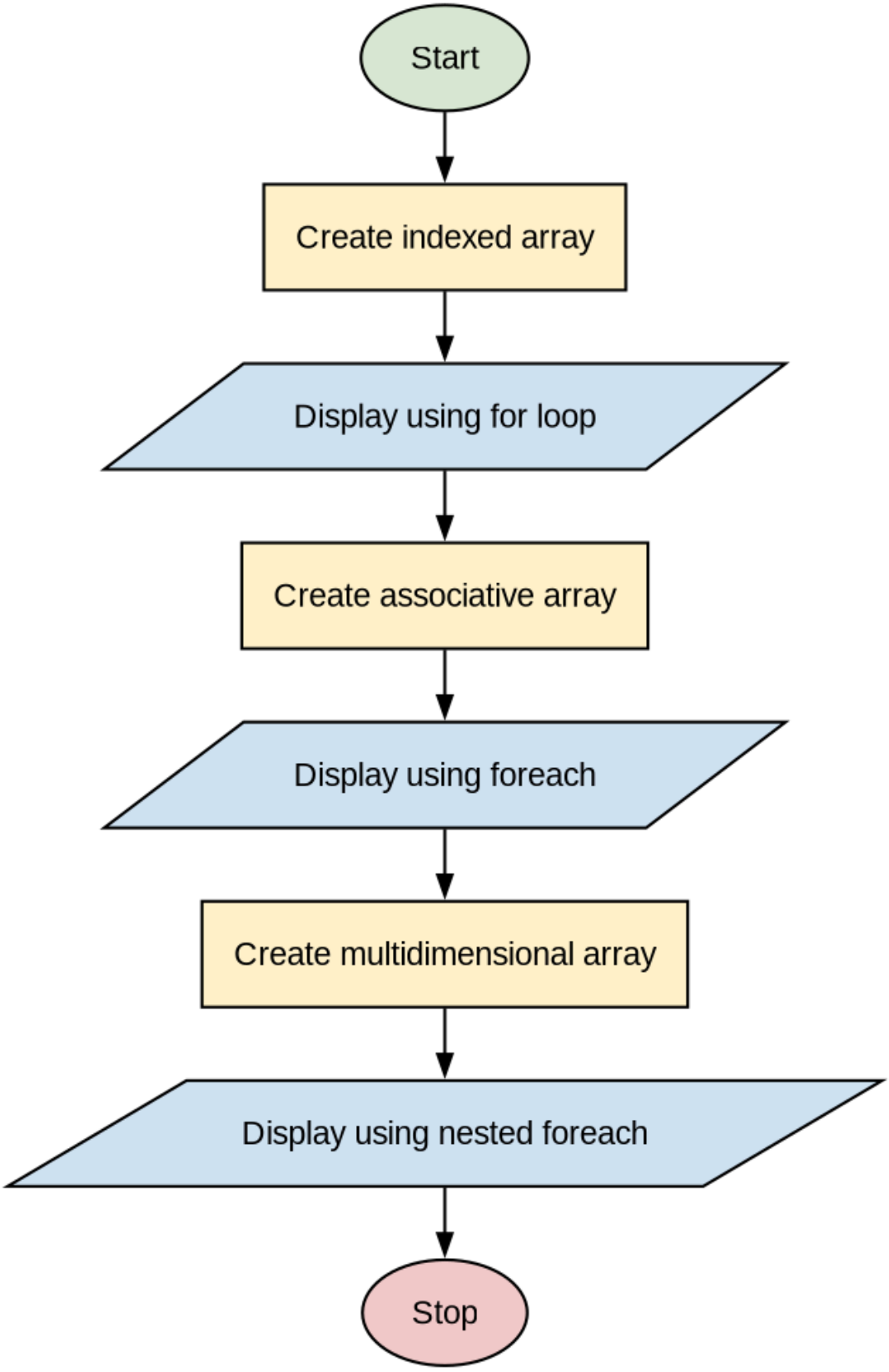
Practical No. 9

Aim: Write a program in PHP to demonstrate the Types of Arrays.

Algorithm:

Step 1: Start.
Step 2: Create an indexed array $colors and display its elements using a for loop.
Step 3: Create an associative array $marks (key => value) and display it using foreach.
Step 4: Create a multidimensional array $students and display it using nested foreach.
Step 5: Stop.

Flowchart:


Start
Create indexed array
Display using for loop
Create associative array
Display using foreach
Create multidimensional array
Display using nested foreach
Stop

Program:

```
<?php
// 1. Indexed Array
$colors = array("Red", "Green", "Blue");
echo "<h3>1. Indexed Array</h3>";
for ($i = 0; $i < count($colors); $i++) {
    echo "colors[$i] = " . $colors[$i] . "<br>";
}

// 2. Associative Array
$marks = array("Rahul" => 82, "Priya" => 90, "Amit" => 76);
echo "<h3>2. Associative Array</h3>";
foreach ($marks as $name => $m) {
    echo "$name => $m <br>";
}

// 3. Multidimensional Array
$students = array(
    array("Rahul", "MCA", 82),
    array("Priya", "MCA", 90),
    array("Amit",  "MBA", 76)
);
echo "<h3>3. Multidimensional Array</h3>";
foreach ($students as $row) {
    foreach ($row as $value) {
        echo $value . " ";
    }
    echo "<br>";
}
?>
```

Output:

```
1. Indexed Array
colors[0] = Red
colors[1] = Green
colors[2] = Blue

2. Associative Array
Rahul => 82
Priya => 90
Amit => 76

3. Multidimensional Array
Rahul MCA 82
Priya MCA 90
Amit MBA 76
```

Practical No. 10

Aim: Write a program in PHP to search an element in an array.

Algorithm:

Step 1: Start.

Step 2: Initialize the array $arr and the element to search $key.

Step 3: Set found = false and i = 0.

Step 4: Repeat while i < count($arr):

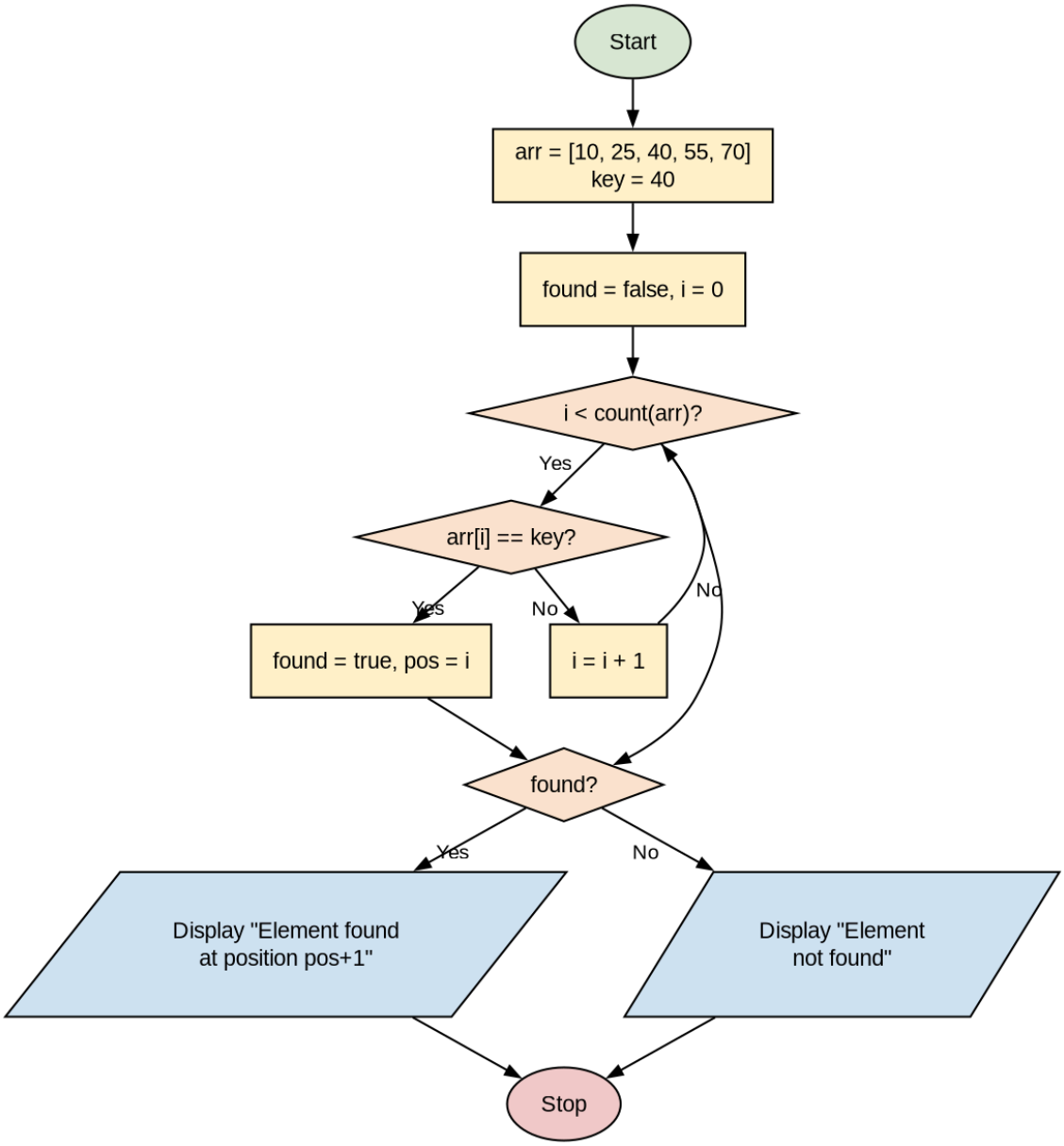
If $arr[i] == $key then set found = true, pos = i and exit the loop.

Otherwise i = i + 1.

Step 5: If found = true display "Element found at position pos + 1", else display "Element not found".

Step 6: Stop.

Flowchart:


Start
arr = [10, 25, 40, 55, 70]
key = 40
found = false, i = 0
i < count(arr)?
Yes
arr[i] == key?
Yes
No
No
found = true, pos = i
i = i + 1
found?
Yes
No
Display "Element found
at position pos+1"
Display "Element
not found"
Stop

Program:

```
<?php
$arr = array(10, 25, 40, 55, 70);
$key = 40;
$found = false;

echo "Array: " . implode(", ", $arr) . "<br>";
echo "Element to search: $key <br><br>";

// Linear search
for ($i = 0; $i < count($arr); $i++) {
    if ($arr[$i] == $key) {
        $found = true;
        $pos = $i;
        break;
    }
}

if ($found) {
    echo "Element $key found at position " . ($pos + 1) . " (index $pos)";
} else {
    echo "Element $key not found in the array";
}

// Using built-in function
echo "<br>Using array_search(): index = " . array_search($key, $arr);
?>
```

Output:

```
Array: 10, 25, 40, 55, 70
Element to search: 40

Element 40 found at position 3 (index 2)
Using array_search(): index = 2
```

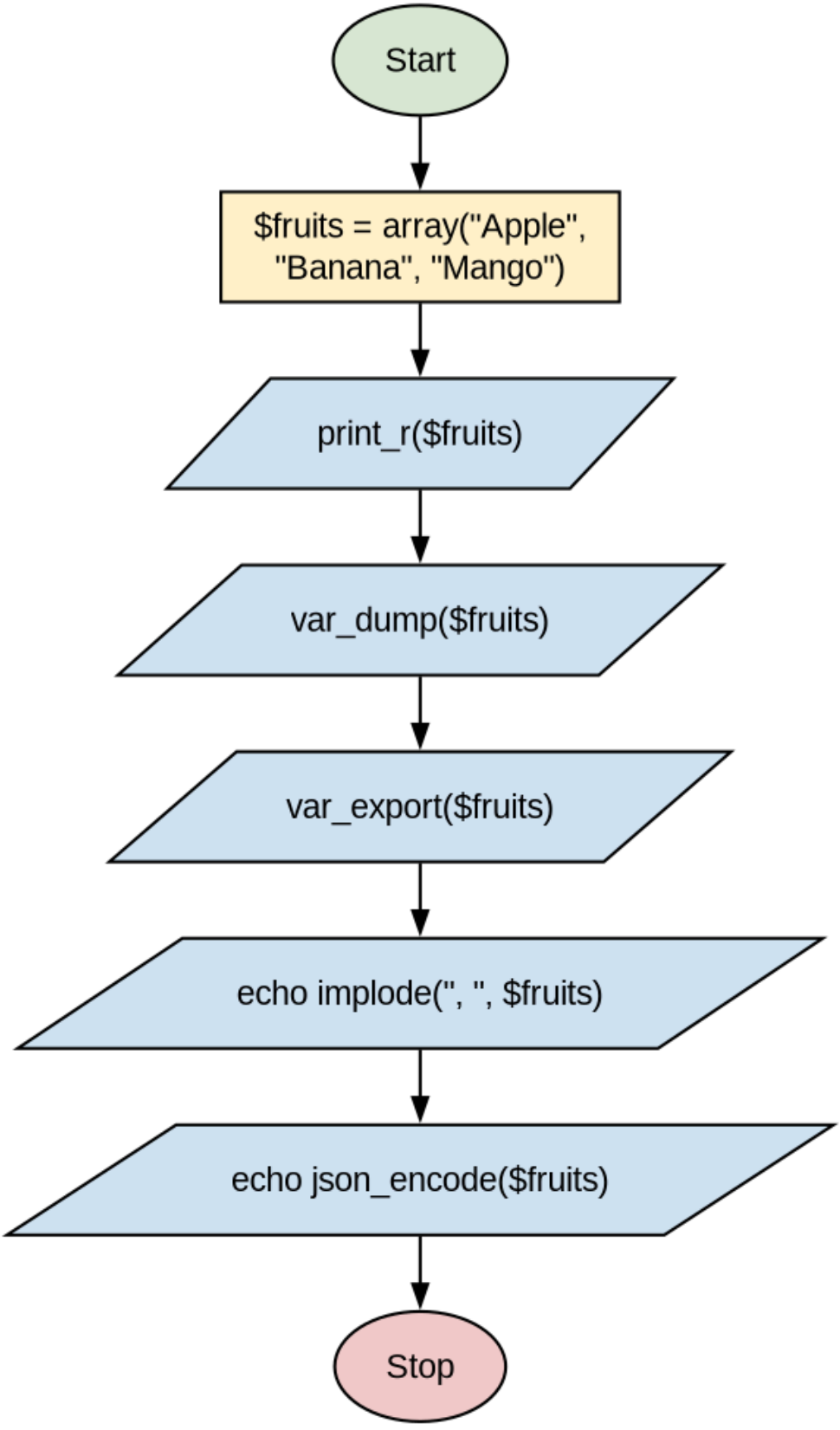
Practical No. 11

Aim: Write a program in PHP to create and print the elements of an array using different Array Printing functions.

Algorithm:

Step 1: Start.
Step 2: Create an array $fruits with elements Apple, Banana, Mango.
Step 3: Print the array using print_r().
Step 4: Print the array using var_dump().
Step 5: Print the array using var_export().
Step 6: Print the array using implode() with echo.
Step 7: Print the array using json_encode().
Step 8: Stop.

Flowchart:


Start
$fruits = array("Apple", "Banana", "Mango")
print_r($fruits)
var_dump($fruits)
var_export($fruits)
echo implode(", ", $fruits)
echo json_encode($fruits)
Stop

Program:

```
<?php
$fruits = array("Apple", "Banana", "Mango");

echo "<pre>";
echo "1. print_r():\n";
print_r($fruits);

echo "\n2. var_dump():\n";
var_dump($fruits);

echo "\n3. var_export():\n";
var_export($fruits);

echo "\n\n4. implode():\n";
echo implode(", ", $fruits);

echo "\n\n5. json_encode():\n";
echo json_encode($fruits);
echo "</pre>";
?>
```

Output:

```
1. print_r():
Array
(
    [0] => Apple
    [1] => Banana
    [2] => Mango
)

2. var_dump():
array(3) {
  [0]=>
  string(5) "Apple"
  [1]=>
  string(6) "Banana"
  [2]=>
  string(5) "Mango"
}

3. var_export():
array (
  0 => 'Apple',
  1 => 'Banana',
  2 => 'Mango',
)

4. implode():
Apple, Banana, Mango

5. json_encode():
["Apple","Banana","Mango"]
```

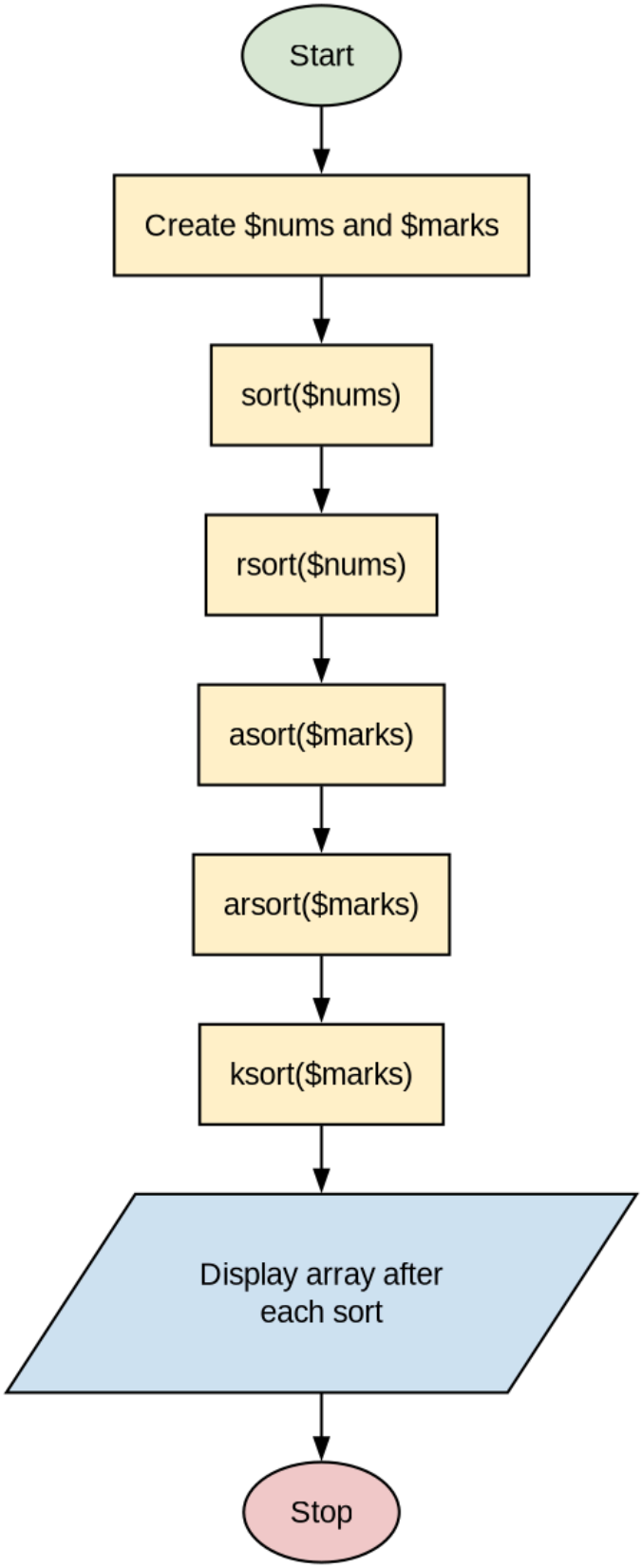
Practical No. 12

Aim: Write a program in PHP that Sorts the given Array using different array sorting functions.

Algorithm:

Step 1: Start.
Step 2: Create an indexed array $nums and an associative array $marks.
Step 3: Sort $nums in ascending order using sort() and display it.
Step 4: Sort $nums in descending order using rsort() and display it.
Step 5: Sort $marks by value in ascending order using asort() and display it.
Step 6: Sort $marks by value in descending order using arsort() and display it.
Step 7: Sort $marks by key using ksort() and display it.
Step 8: Stop.

Flowchart:


Start
Create $nums and $marks
sort($nums)
rsort($nums)
asort($marks)
arsort($marks)
ksort($marks)
Display array after each sort
Stop

Program:

```
<?php
function show($arr) {
    foreach ($arr as $k => $v) {
        echo "[$k] => $v   ";
    }
    echo "<br><br>";
}

$nums  = array(40, 10, 50, 20, 30);
$marks = array("Rahul" => 82, "Priya" => 90, "Amit" => 76);

echo "Original nums: ";  show($nums);
echo "Original marks: "; show($marks);

sort($nums);
echo "1. sort() - ascending: "; show($nums);

rsort($nums);
echo "2. rsort() - descending: "; show($nums);

asort($marks);
echo "3. asort() - by value ascending: "; show($marks);

arsort($marks);
echo "4. arsort() - by value descending: "; show($marks);

ksort($marks);
echo "5. ksort() - by key: "; show($marks);
?>
```

Output:

```
Original nums: [0] => 40  [1] => 10  [2] => 50  [3] => 20  [4] => 30
Original marks: [Rahul] => 82  [Priya] => 90  [Amit] => 76

1. sort() - ascending: [0] => 10  [1] => 20  [2] => 30  [3] => 40  [4] => 50

2. rsort() - descending: [0] => 50  [1] => 40  [2] => 30  [3] => 20  [4] => 10

3. asort() - by value ascending: [Amit] => 76  [Rahul] => 82  [Priya] => 90

4. arsort() - by value descending: [Priya] => 90  [Rahul] => 82  [Amit] => 76

5. ksort() - by key: [Amit] => 76  [Priya] => 90  [Rahul] => 82
```

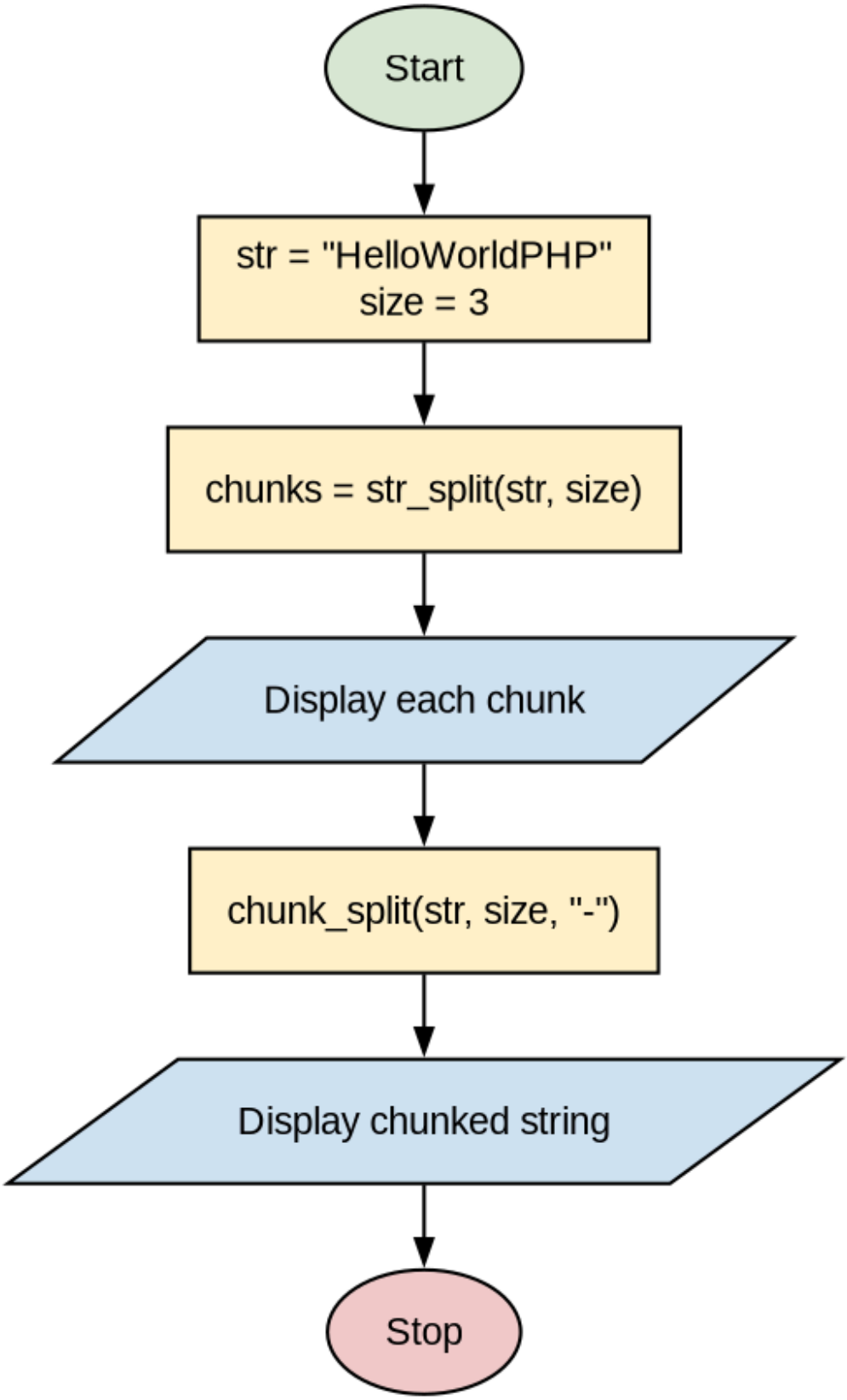
Practical No. 13

Aim: Write a program in PHP that Chunkify the given string.

Algorithm:

Step 1: Start.
Step 2: Initialize the string $str and the chunk size $size.
Step 3: Split the string into an array of chunks using str_split($str, $size).
Step 4: Display each chunk using foreach.
Step 5: Display the chunked string using chunk_split($str, $size, "-").
Step 6: Stop.

Flowchart:


Start
str = "HelloWorldPHP"
size = 3
chunks = str_split(str, size)
Display each chunk
chunk_split(str, size, "-")
Display chunked string
Stop

Program:

```
<?php
$str  = "HelloWorldPHP";
$size = 3;

echo "Original String: $str <br>";
echo "Chunk Size: $size <br><br>";

// Method 1: str_split() returns an array of chunks
$chunks = str_split($str, $size);
echo "Chunks using str_split():<br>";
foreach ($chunks as $i => $c) {
    echo "Chunk " . ($i + 1) . ": $c <br>";
}

// Method 2: chunk_split() returns a string with separator
echo "<br>Using chunk_split(): " . chunk_split($str, $size, "-");
?>
```

Output:

```
Original String: HelloWorldPHP
Chunk Size: 3

Chunks using str_split():
Chunk 1: Hel
Chunk 2: loW
Chunk 3: orl
Chunk 4: dPH
Chunk 5: P

Using chunk_split(): Hel-loW-orl-dPH-P-
```

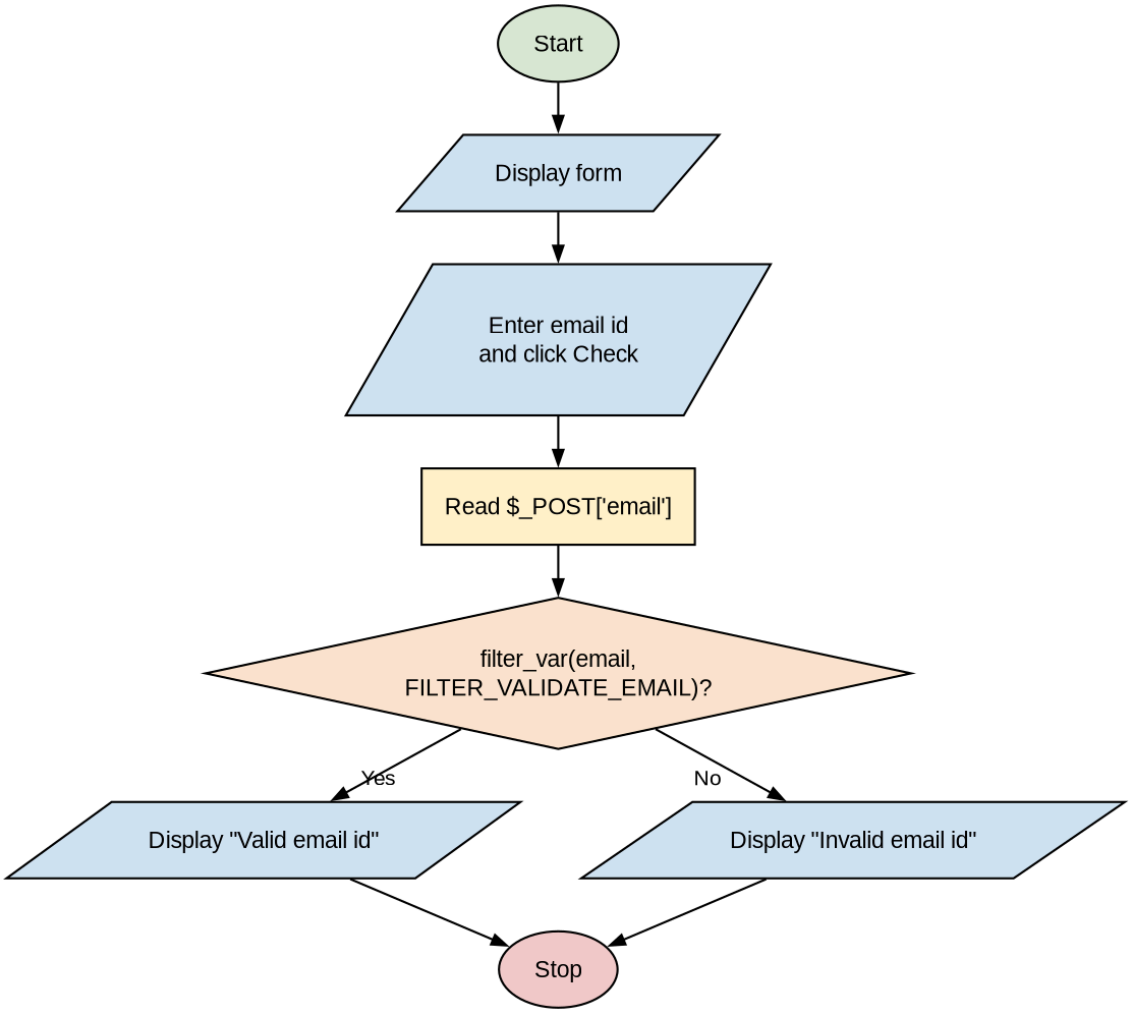
Practical No. 14

Aim: Write a PHP script to check that email id is valid or not.

Algorithm:

Step 1: Start.
Step 2: Display a form to enter an email id.
Step 3: User enters the email id and clicks Check.
Step 4: Read the email from $_POST['email'].
Step 5: Validate it using filter_var($email, FILTER_VALIDATE_EMAIL).
Step 6: If valid display "is a valid email id", else display "is NOT a valid email id".
Step 7: Stop.

Flowchart:


Start
Display form
Enter email id
and click Check
Read $_POST['email']
filter_var(email,
FILTER_VALIDATE_EMAIL)?
Yes
No
Display "Valid email id"
Display "Invalid email id"
Stop

Program:

```
<html>
<body>
<h2>Email Validation</h2>
<form method="post">
  Enter Email: <input type="text" name="email">
  <input type="submit" name="check" value="Check">
</form>

<?php
if (isset($_POST['check'])) {
    $email = trim($_POST['email']);

    if (filter_var($email, FILTER_VALIDATE_EMAIL)) {
        echo "<p style='color:green'>$email is a valid email id.</p>";
    } else {
        echo "<p style='color:red'>$email is NOT a valid email id.</p>";
    }
}
?>
</body>
</html>
```

Output:

```
Email Validation
Enter Email: [rahul@gmail.com    ] [Check]
rahul@gmail.com is a valid email id.

Enter Email: [rahul.gmail.com    ] [Check]
rahul.gmail.com is NOT a valid email id.
```

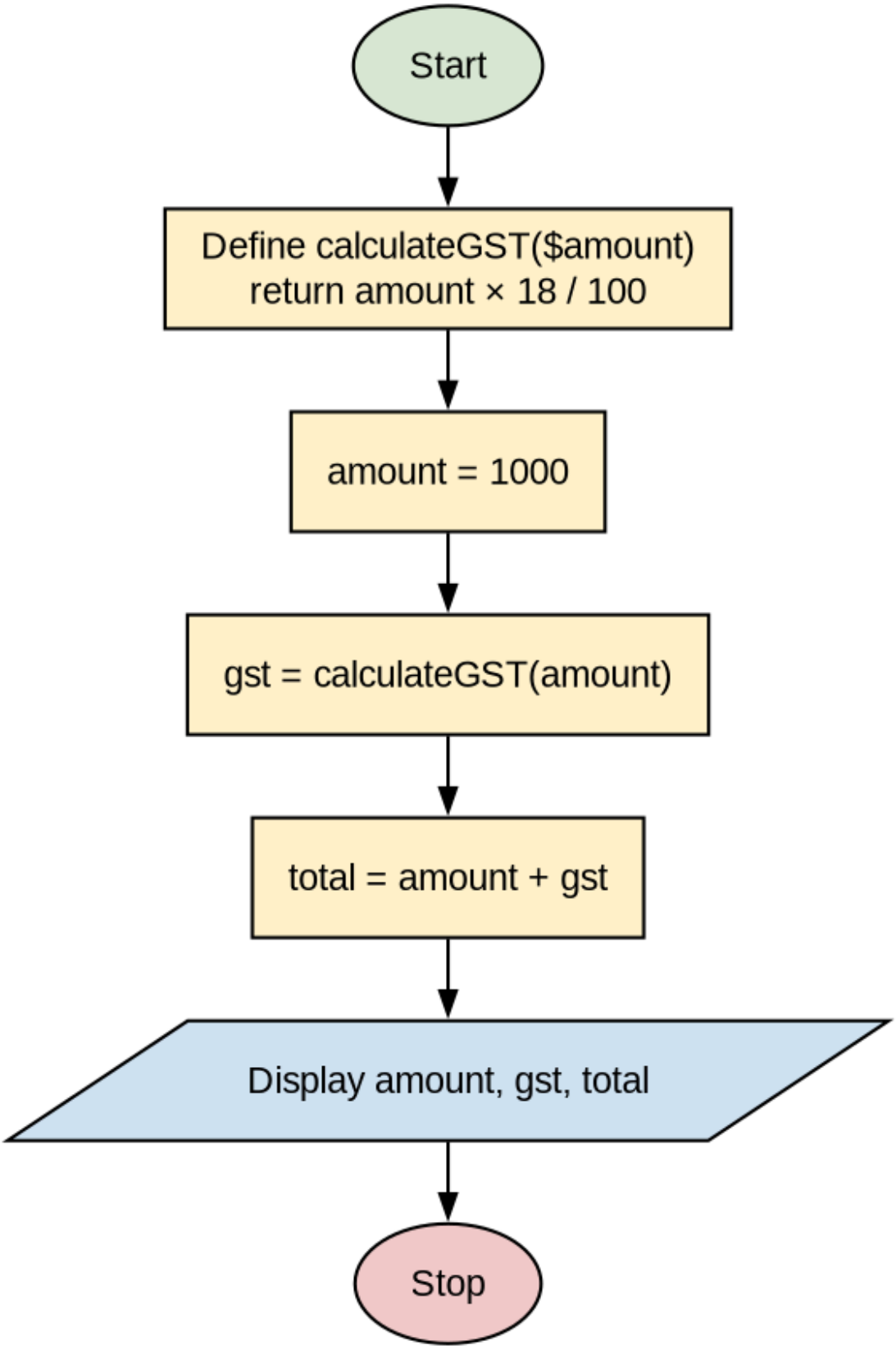
Practical No. 15

Aim: Write a PHP function calculateGST($amount) that returns the GST (18%) of the given amount. Call the function with a sample value and display the result.

Algorithm:

Step 1: Start.
Step 2: Define function calculateGST($amount) which returns $amount * 18 / 100.
Step 3: Set $amount = 1000.
Step 4: Call $gst = calculateGST($amount).
Step 5: Calculate $total = $amount + $gst.
Step 6: Display Amount, GST and Total Amount.
Step 7: Stop.

Flowchart:


Start
Define calculateGST($amount)
return amount × 18 / 100
amount = 1000
gst = calculateGST(amount)
total = amount + gst
Display amount, gst, total
Stop

Program:

```
<?php
function calculateGST($amount) {
    $gst = $amount * 18 / 100;
    return $gst;
}

$amount = 1000;
$gst    = calculateGST($amount);
$total  = $amount + $gst;

echo "<h3>GST Calculation</h3>";
echo "Amount       : Rs. $amount <br>";
echo "GST (18%)    : Rs. $gst <br>";
echo "Total Amount : Rs. $total <br>";
?>
```

Output:

```
GST Calculation
Amount       : Rs. 1000
GST (18%)    : Rs. 180
Total Amount : Rs. 1180
```